排序算法

算法算法，计算方法

Rosaya

目录

1. 算法 2
   1.1 算法定义 3
   1.2 算法效率 4
   1.3 习题 7
2. 简单排序 8
   2.1 排序简介 9
   2.2 选择排序 10
   2.3 冒泡排序 12
   2.4 插入排序 15
   2.5 计数排序 16
3. 复杂排序 18
   3.1 桶排序 19
   3.2 希尔排序 21
   3.3 归并排序 24
   3.4 快速排序 27
   3.5 基数排序 33
   3.6 习题 35

目录

1. 算法 2
   1.1 算法定义 3
   1.2 算法效率 4
   1.3 习题 7
2. 简单排序 8
   2.1 排序简介 9
   2.2 选择排序 10
   2.3 冒泡排序 12
   2.4 插入排序 15
   2.5 计数排序 16
3. 复杂排序 18
   3.1 桶排序 19
   3.2 希尔排序 21
   3.3 归并排序 24
   3.4 快速排序 27
   3.5 基数排序 33
   3.6 习题 35

1.1 算法定义

1.1 算法定义

算法，顾名思义，即计算的方法。

算法通常用于解决特定的计算任务，但与可以直接在计算机上运行的程序不同，算法使用数学化的描述，更加侧重于思想，可以被看作抽象的程序。

同一个算法可以有许多种不同的实现方式，两个不同的程序里也可能使用了同一种算法。

1.1 算法定义

算法，顾名思义，即计算的方法。

算法通常用于解决特定的计算任务，但与可以直接在计算机上运行的程序不同，算法使用数学化的描述，更加侧重于思想，可以被看作抽象的程序。

同一个算法可以有许多种不同的实现方式，两个不同的程序里也可能使用了同一种算法。

在 OI 中一般学习的是**确定性算法**，即运行的结果与中间执行的过程在**相同的输入下完全相同**。

而现在学术界研究更多的则是**随机化算法**，一般随机化算法**具有更高的效率**与**更广泛的应用范围**。

除了**确定/随机**，算法还有**在线/离线**，**动态/静态**等等分类。

1.1 算法定义

算法，顾名思义，即计算的方法。

算法通常用于解决特定的计算任务，但与可以直接在计算机上运行的程序不同，算法使用数学化的描述，更加侧重于思想，可以被看作抽象的程序。

同一个算法可以有许多种不同的实现方式，两个不同的程序里也可能使用了同一种算法。

在 OI 中一般学习的是**确定性算法**，即运行的结果与中间执行的过程在**相同的输入下完全相同**。

而现在学术界研究更多的则是**随机化算法**，一般随机化算法**具有更高的效率**与**更广泛的应用范围**。

除了**确定/随机**，算法还有**在线/离线**，**动态/静态**等等分类。

算法可视化网站，非常好用，有助于理解算法。

1.2 算法效率

1.2 算法效率

我们用复杂度**衡量算法的效率**。

包括**时间复杂度**和**空间复杂度**两个方面，这里主要讨论时间复杂度。

1.2 算法效率

我们用复杂度**衡量算法的效率**。

包括**时间复杂度**和**空间复杂度**两个方面，这里主要讨论时间复杂度。

```
FIB(n):
if n < 0:
    return null
if n = 0 or n = 1:                    // 1
    return n
return FIB(n − 1) + FIB(n − 2)        // 2
```

1.2 算法效率

我们用复杂度**衡量算法的效率**。

包括**时间复杂度**和**空间复杂度**两个方面，这里主要讨论时间复杂度。

```
FIB(n):
if n < 0:
    return null
if n = 0 or n = 1:                    // 1
    return n
return FIB(n − 1) + FIB(n − 2)        // 2
```

我们用运行次数 $T(n)$ 表示**基本操作**的**执行次数**，显然 $T(n) = 3 + T(n-1) + T(n-2)$，边界情况为 $T(0) = T(1) = 2$。

但是**数清基本操作的数量**是困难的，此时我们引入**渐进记号**来描述复杂度。

1.2 算法效率

1.2 算法效率

💡 定义 1.1 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

1.2 算法效率

💡 定义 1.1 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 =$$

1.2 算法效率

💡 定义 1.1 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 = \Theta(n^2)$$

1.2 算法效率

💡 定义 1.1 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 = \Theta(n^2)$$

$$n\sqrt{n} + n\log^5 n + m\log m + nm =$$

1.2 算法效率

💡 **定义 1.1** 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 = \Theta(n^2)$$

$$n\sqrt{n} + n\log^5 n + m\log m + nm = \Theta\left(n\sqrt{n} + m\log m + nm\right)$$

1.2 算法效率

💡 **定义 1.1** 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 = \Theta(n^2)$$

$$n\sqrt{n} + n\log^5 n + m\log m + nm = \Theta\left(n\sqrt{n} + m\log m + nm\right)$$

这里要注意，我们使用渐进记号分析复杂度的时候其实忽略了**常数项**，但常数项往往对**实际问题的分析**是非常关键的。

1.2 算法效率

💡 **定义 1.1** 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = \Theta(g(n))$ 当且仅当 $\exists c_1, c_2, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$。

也就是说 $\Theta(g(n))$ 是 n **充分大**时构成 $f(n)$ 的最主要的项。一般称 Θ 为复杂度的严格分析，只考虑基本操作数量的**阶**（一般可以表示增长速度）。

$$3n^2 + 5n - 3 = \Theta(n^2)$$

$$n\sqrt{n} + n\log^5 n + m\log m + nm = \Theta\left(n\sqrt{n} + m\log m + nm\right)$$

这里要注意，我们使用渐进记号分析复杂度的时候其实忽略了**常数项**，但常数项往往对**实际问题的分析**是非常关键的。

$$10^9 = \Theta(1)$$

$$n = \Theta(n)$$

1.2 算法效率

1.2 算法效率

💡 定义 1.2 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = O(g(n))$ 当且仅当 $\exists c, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \leq f(n) \leq cg(n)$。

研究时间复杂度时通常会使用 O 符号，因为我们关注的通常是**程序用时的上界**，而**不关心其用时的下界**。

由此我们给出复杂度的定义：若**最坏情况下**基本操作执行次数 $T(n) = O(f(n))$，则算法复杂度为 $O(f(n))$。当然我们希望复杂度尽量优秀，所以我们应该取**阶最低**的满足 $T(n) = O(f(n))$ 的 $f(n)$。

1.2 算法效率

💡 定义 1.2 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = O(g(n))$ 当且仅当 $\exists c, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \leq f(n) \leq cg(n)$。

研究时间复杂度时通常会使用 O 符号，因为我们关注的通常是**程序用时的上界**，而**不关心其用时的下界**。

由此我们给出复杂度的定义：若**最坏情况下**基本操作执行次数 $T(n) = O(f(n))$，则算法复杂度为 $O(f(n))$。当然我们希望复杂度尽量优秀，所以我们应该取**阶最低**的满足 $T(n) = O(f(n))$ 的 $f(n)$。

$$10^9 = O(1)$$

1.2 算法效率

💡 定义 1.2 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = O(g(n))$ 当且仅当 $\exists c, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le f(n) \le cg(n)$。

研究时间复杂度时通常会使用 O 符号，因为我们关注的通常是**程序用时的上界**，而**不关心其用时的下界**。

由此我们给出复杂度的定义：若**最坏情况下**基本操作执行次数 $T(n) = O(f(n))$，则算法复杂度为 $O(f(n))$。当然我们希望复杂度尽量优秀，所以我们应该取**阶最低**的满足 $T(n) = O(f(n))$ 的 $f(n)$。

$$10^9 = O(1) \qquad \checkmark$$

$$10^9 = O(n)$$

1.2 算法效率

💡 定义 1.2 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = O(g(n))$ 当且仅当 $\exists c, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le f(n) \le cg(n)$。

研究时间复杂度时通常会使用 O 符号，因为我们关注的通常是**程序用时的上界**，而**不关心其用时的下界**。

由此我们给出复杂度的定义：若**最坏情况下**基本操作执行次数 $T(n) = O(f(n))$，则算法复杂度为 $O(f(n))$。当然我们希望复杂度尽量优秀，所以我们应该取**阶最低**的满足 $T(n) = O(f(n))$ 的 $f(n)$。

$$10^9 = O(1) \qquad \checkmark$$

$$10^9 = O(n) \qquad \times$$

$$10^9 = O(n^2 + n)$$

1.2 算法效率

💡 定义 1.2 对于函数 $f(n)$ 和 $g(n)$，我们称 $f(n) = O(g(n))$ 当且仅当 $\exists c, n_0 > 0$，使得 $\forall n > n_0$，都有 $0 \le f(n) \le cg(n)$。

研究时间复杂度时通常会使用 O 符号，因为我们关注的通常是**程序用时的上界**，而**不关心其用时的下界**。

由此我们给出复杂度的定义：若**最坏情况下**基本操作执行次数 $T(n) = O(f(n))$，则算法复杂度为 $O(f(n))$。当然我们希望复杂度尽量优秀，所以我们应该取**阶最低**的满足 $T(n) = O(f(n))$ 的 $f(n)$。

$$
\begin{aligned}
10^9 &= O(1) && \checkmark \\
10^9 &= O(n) && \times \\
10^9 &= O(n^2 + n) && \times \\
&\vdots
\end{aligned}
$$

1.3 习题

1.3 习题

通过渐进记号的定义证明以下算法的时间复杂度。

1.3 习题

通过渐进记号的定义证明以下算法的时间复杂度。

```
GCD(a, b):
if b = 0:
    return a
return GCD(b, a mod b)
```

1.3 习题

通过渐进记号的定义证明以下算法的时间复杂度。

```
GCD(a, b):
if b = 0:
    return a
return GCD(b, a mod b)
```

```
COUNTINGFACTOR(n):
let cnt ← 0
for i from 1 to n:
    for j from 1 to n/i:
        cnt ← cnt + 1
return cnt
```

目录

1. 算法 2
   1.1 算法定义 3
   1.2 算法效率 4
   1.3 习题 7
2. 简单排序 8
   2.1 排序简介 9
   2.2 选择排序 10
   2.3 冒泡排序 12
   2.4 插入排序 15
   2.5 计数排序 16
3. 复杂排序 18
   3.1 桶排序 19
   3.2 希尔排序 21
   3.3 归并排序 24
   3.4 快速排序 27
   3.5 基数排序 33
   3.6 习题 35

2.1 排序简介

2.1 排序简介

排序算法（Sorting algorithm）是一种将一组特定的数据按某种顺序（例如从小到大）进行排列的算法。排序算法多种多样，性质也大多不同。

2.1 排序简介

排序算法（Sorting algorithm）是一种将一组特定的数据按某种顺序（例如从小到大）进行排列的算法。排序算法多种多样，性质也大多不同。

- **正确性**

排序算法**必要条件**。

- **时空复杂度**

算法运行的**时间效率**和运行**花费的额外空间**。

- **稳定性**

稳定性是指相等的元素经过排序之后相对顺序是否发生了改变。拥有稳定性这一特性的算法会让原本有相等键值的纪录**维持相对次序**，即如果一个排序算法是稳定的，当有两个相等键值的纪录 R 和 S，且在原本的列表中 R 出现在 S 之前，在排序过的列表中 R 也将会是在 S 之前。

2.2 选择排序

2.2 选择排序

所以，我们怎么排序？

一个简单的想法就是每次选择**最小值**排成一个新的序列，这样对于 $\forall 1 \leq i < n, A_i \leq A_{i+1}$，最终序列升序。

2.2 选择排序

所以，我们怎么排序？

一个简单的想法就是每次选择**最小值**排成一个新的序列，这样对于 $\forall 1 \le i < n, A_i \le A_{i+1}$，最终序列升序。

```
SelectionSort(n, A[1...n]):
for i from 1 to n:
    let p ← i
    for j from i + 1 to n:
        if A[j] < A[p]:
            p ← j
    swap A[i] and A[p]
return A[1...n]
```

2.2 选择排序

所以，我们怎么排序？

一个简单的想法就是每次选择**最小值**排成一个新的序列，这样对于 $\forall 1 \le i < n, A_i \le A_{i+1}$，最终序列升序。

```
SelectionSort(n, A[1...n]):
for i from 1 to n:
    let p ← i
    for j from i + 1 to n:
        if A[j] < A[p]:
            p ← j
    swap A[i] and A[p]
return A[1...n]
```

正确性，**稳定性**，**时空复杂度**，如何分析？

2.2 选择排序

2.2 选择排序

稳定性优化:

1. 使用额外数组记录最终排序答案，额外空间变为 $O(n)$，但算法变为稳定排序。
2. 使用链表模拟过程，避免中间出现的交换操作，算法变为稳定排序。

2.2 选择排序

稳定性优化：

1. 使用额外数组记录最终排序答案，额外空间变为 $O(n)$，但算法变为稳定排序。
2. 使用链表模拟过程，避免中间出现的交换操作，算法变为稳定排序。

时间复杂度优化：

1. 使用线段树等数据结构完成每次取**最小值**的操作，额外空间变为 $O(n)$，但时间降至 $O(n \log n)$。
2. 使用堆完成每次取**全局最小值**的操作，时间降至 $O(n \log n)$。

2.3 冒泡排序

2.3 冒泡排序

选择排序的不稳定性源于可能会**交换两个不相邻的数**。

那么有没有一种只会交换相邻的数的排序算法呢？

2.3 冒泡排序

选择排序的不稳定性源于可能会**交换两个不相邻的数**。

那么有没有一种只会交换相邻的数的排序算法呢？

考虑插入排序找到**最大值后移**的操作，那么我们对最大值左侧的数完全没有处理，那为何不在左侧也做一个类似于选取最大值移动过来的操作呢？

2.3 冒泡排序

选择排序的不稳定性源于可能会**交换两个不相邻的数**。

那么有没有一种只会交换相邻的数的排序算法呢？

考虑插入排序找到**最大值后移**的操作，那么我们对最大值左侧的数完全没有处理，那为何不在左侧也做一个类似于选取最大值移动过来的操作呢？

```
BubbleSort(n, A[1...n]):
for i from 1 to n:
    for j from 1 to n − 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
return A[1...n]
```

2.3 冒泡排序

选择排序的不稳定性源于可能会**交换两个不相邻的数**。

那么有没有一种只会交换相邻的数的排序算法呢？

考虑插入排序找到**最大值后移**的操作，那么我们对最大值左侧的数完全没有处理，那为何不在左侧也做一个类似于选取最大值移动过来的操作呢？

```
BubbleSort(n, A[1...n]):
for i from 1 to n:
    for j from 1 to n − 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
return A[1...n]
```

这就是冒泡排序，最小值像气泡一样**浮到顶端**。**正确性**，**稳定性**，**时空复杂度**，如何分析？

2.3 冒泡排序

2.3 冒泡排序

冒泡排序的交换次数，与**逆序对数**相同。

$$\mathrm{inv}(p) := \#\{A_i > A_j \wedge i < j\}$$

每次交换操作都**恰好消除一个逆序对**，所以总复杂度一定是 $O(n^2)$。

2.3 冒泡排序

冒泡排序的交换次数，与**逆序对数**相同。

$$\mathrm{inv}(p) := \#\{A_i > A_j \wedge i < j\}$$

每次交换操作都**恰好消除一个逆序对**，所以总复杂度一定是 $O(n^2)$。

冒泡排序在**多少轮后**序列有序？

2.3 冒泡排序

2.3 冒泡排序

那下文所示的**鸡尾酒排序**算法呢？

```
CocktailSort(n, A[1...n]):
for i from 1 to n:
    for j from 1 to n − 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
    for j from n − 1 to 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
return A[1...n]
```

2.3 冒泡排序

那下文所示的**鸡尾酒排序**算法呢？

```
CocktailSort(n, A[1...n]):
for i from 1 to n:
    for j from 1 to n − 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
    for j from n − 1 to 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
return A[1...n]
```

但由于**逆序对数**作为时间复杂度下界，算法效率一定为 $O(n^2)$！

2.4 插入排序

2.4 插入排序

考虑如果我们已经排好了 $A_1, A_2, \ldots, A_i$，如何把 A_{i+1} 排到应该排的位置上？

2.4 插入排序

考虑如果我们已经排好了 $A_1, A_2, \ldots, A_i$，如何把 A_{i+1} 排到应该排的位置上？

通过与相邻元素**交换至目标位置**完成对新元素的**插入**。

```
InsertionSort(n, A[1...n]):
for i from 1 to n:
    for j from i − 1 to 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
        else:
            break
return A[1...n]
```

2.4 插入排序

考虑如果我们已经排好了 $A_1, A_2, \ldots, A_i$，如何把 A_{i+1} 排到应该排的位置上？

通过与相邻元素**交换至目标位置**完成对新元素的**插入**。

```
InsertionSort(n, A[1...n]):
for i from 1 to n:
    for j from i − 1 to 1:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
        else:
            break
return A[1...n]
```

老样子，**正确性**，**稳定性**，**时空复杂度**，如何分析？

依然可以使用平衡树等数据结构优化维持有序序列的操作。

2.5 计数排序

2.5 计数排序

如果我们只需要排整数的话，为什么**一定要比较两个数的大小呢**？

在接下来的章节我们会证明，基于**比较**的排序算法在所有情况下的**平均复杂度**不会低于 $O(n \log n)$。

2.5 计数排序

如果我们只需要排整数的话，为什么**一定要比较两个数的大小呢**？

在接下来的章节我们会证明，基于**比较**的排序算法在所有情况下的**平均复杂度**不会低于 $O(n \log n)$。

如果数的值域只有 $V = O(n)$，我们可以开一个大小为 V 的数组来统计**每个数的出现次数**。为了保持**稳定性**，可以参考以下实现方式。

2.5 计数排序

2.5 计数排序

```
CountingSort(n, A[1...n]):
let B[1...n] ← {}
let C[1...V] ← {}
for i from 1 to n:
    C[A[i]] ← C[A[i]] + 1
for i from 1 to V:
    C[i] ← C[i] + C[i − 1]
for i from n to 1:
    B[C[A[i]]] ← A[i]
    C[A[i]] ← C[A[i]] − 1
return B[1...n]
```

老三样？

目录

1. 算法 2
1.1 算法定义 3
1.2 算法效率 4
1.3 习题 7
2. 简单排序 8
2.1 排序简介 9
2.2 选择排序 10
2.3 冒泡排序 12
2.4 插入排序 15
2.5 计数排序 16
3. 复杂排序 18
3.1 桶排序 19
3.2 希尔排序 21
3.3 归并排序 24
3.4 快速排序 27
3.5 基数排序 33
3.6 习题 35

3.1 桶排序

3.1 桶排序

以上算法较为基础，但难以优化到更好的时间复杂度。

但是有一个共同的优点，就是**常数很小**。

如果我们有方法**降低数据规模**，那排序效率实际非常高。

3.1 桶排序

以上算法较为基础，但难以优化到更好的时间复杂度。

但是有一个共同的优点，就是**常数很小**。

如果我们有方法**降低数据规模**，那排序效率实际非常高。

所以我们考虑按某些值域将所有元素**预分组**，然后组内再进行排序。

```
BucketSort(n, m, A[1...n]):
let B[1...n] ← {}
for i from 1 to n:
    B[i] = A[i]/⌈n/m⌉
CountingSort(n, B[1...n])
for i from 0 to m:
    InsertionSort(C[i] − C[i − 1], A[C[i − 1] + 1...C[i]])
return A[1...n]
```

3.1 桶排序

3.1 桶排序

老三样？

这里我们分析的复杂度是**平均**复杂度，也就是在所有数据下的平均效率。

3.1 桶排序

老三样？

这里我们分析的复杂度是**平均**复杂度，也就是在所有数据下的平均效率。

可以看到平均意义下相当于**每一段数量平均**，那么复杂度为 $O\left(n+m\left(\frac{n}{m}\right)^2+m\right)=O\left(n+\frac{n^2}{m}+m\right)$。

平衡复杂度，当 $m=O(n)$ 时，复杂度为 $O(n)$。

3.2 希尔排序

3.2 希尔排序

希尔排序以它的发明者希尔（Donald Shell）命名。

具体想法是每次插入排序**分组**，但分组方式与桶排序不同，选用的是按模数分组，具体方式如下（H 单调递增，$H_1 = 1$）。

3.2 希尔排序

希尔排序以它的发明者希尔（Donald Shell）命名。

具体想法是每次插入排序**分组**，但分组方式与桶排序不同，选用的是按模数分组，具体方式如下（H 单调递增，$H_1 = 1$）。

```
SHELLSORT(n, m, A[1...n], H[1...n]):
for i from m to 1:
    for j from 1 to n:
        let k ← j
        while k − H[i] > 0 ∧ A[k − H[i]] > A[k]:
            swap A[k − H[i]] and A[k]
            k ← k − H[i]
return A[1...n]
```

老三样？

3.2 希尔排序

3.2 希尔排序

显然**复杂度分析**和 H 选取有关，所以本身是没法分析的。

如果取 $H_i = 2^i - 1$，如何分析复杂度？

（提示：考虑**每一轮**中每个数往前移动的步数）

3.2 希尔排序

显然**复杂度分析**和 H 选取有关，所以本身是没法分析的。

如果取 $H_i = 2^i - 1$，如何分析复杂度？

（提示：考虑**每一轮**中每个数往前移动的步数）

显然步长为 H_i 时每个元素最多移动 $\dfrac{n}{H_i}$ 步。

$T(n) = \sum_{i=1}^{m} O\left(\dfrac{n^2}{H_i}\right) = O(n^2)$，显然不是我们**想要的结果**。如果 $H_i < \sqrt{n}$ 的操作复杂度能够更低就好了。

3.2 希尔排序

显然**复杂度分析**和 H 选取有关，所以本身是没法分析的。

如果取 $H_i = 2^i - 1$，如何分析复杂度？

（提示：考虑**每一轮**中每个数往前移动的步数）

显然步长为 H_i 时每个元素最多移动 $\dfrac{n}{H_i}$ 步。

$T(n) = \sum_{i=1}^{m} O\left(\dfrac{n^2}{H_i}\right) = O(n^2)$，显然不是我们**想要的结果**。如果 $H_i < \sqrt{n}$ 的操作复杂度能够更低就好了。

💡 定理 3.1 若曾经进行过步长为 a, b 的插入排序且 $(a, b) = 1$，再进行步长为 c 的插入排序**复杂度**为 $O\left(n\dfrac{ab}{c}\right)$。

3.2 希尔排序

3.2 希尔排序

若**定理正确**则 $T(n) = \sum_{H_i \geq \sqrt{n}} O\left(\frac{n^2}{H_i}\right) + \sum_{H_i < \sqrt{n}} O\left(n \frac{H_{i+1} H_{i+2}}{H_i}\right) = O(n\sqrt{n})$。

3.2 希尔排序

若**定理正确**则 $T(n) = \sum_{H_i \geq \sqrt{n}} O\left(\frac{n^2}{H_i}\right) + \sum_{H_i < \sqrt{n}} O\left(n \frac{H_{i+1} H_{i+2}}{H_i}\right) = O(n\sqrt{n})$。

我们可以用两个**引理**证明**定理**。

💡 引理 3.2 若曾经进行过步长为 a 的插入排序，则之后的任意一轮插入排序结束之后 $\forall 1 \leq i \leq n-a$，均有 $A_i \leq A_{i+a}$。

💡 引理 3.3 给定正整数 a, b，若 $(a, b) = 1$，则无法表示成 a, b 的非负线下组合的最大的正整数是 $ab - a - b$。

尝试由**引理**证明**定理**。

3.2 希尔排序

若**定理正确**则 $T(n) = \sum_{H_i \geq \sqrt{n}} O\left(\frac{n^2}{H_i}\right) + \sum_{H_i < \sqrt{n}} O\left(n \frac{H_{i+1} H_{i+2}}{H_i}\right) = O(n\sqrt{n})$。

我们可以用两个**引理**证明**定理**。

💡 引理 3.2 若曾经进行过步长为 a 的插入排序，则之后的任意一轮插入排序结束之后 $\forall 1 \leq i \leq n-a$，均有 $A_i \leq A_{i+a}$。

💡 引理 3.3 给定正整数 a, b，若 $(a, b) = 1$，则无法表示成 a, b 的非负线下组合的最大的正整数是 $ab - a - b$。

尝试由**引理**证明**定理**。

此外，若取 $H = \{2^p 3^q \leq n | p, q \in \mathbb{N}\}$，从小到大，那么可以得到 $O(n \log^2 n)$ 的复杂度，感兴趣的同学可以证明。

3.3 归并排序

3.3 归并排序

再次回到**插入排序**，考虑它为什么慢。

3.3 归并排序

再次回到**插入排序**，考虑它为什么慢。

我们发现，如果把 1 个数插入到长度为 n 的有序数组中，复杂度就为 $O(n)$ 了，那我们**为什么不同时插入** m 个数呢？

3.3 归并排序

再次回到**插入排序**，考虑它为什么慢。

我们发现，如果把 1 个数插入到长度为 n 的有序数组中，复杂度就为 $O(n)$ 了，那我们**为什么不同时插入** m 个数呢？

这样并不是太好做，但如果这 m 个数**也是有序的**就好了。

```
MERGE(n, m, A[1...n], B[1...m]):
let C[1...n + m] ← {}, i ← 1, j ← 1
for k from 1 to n + m:
    if i ≤ n ∧ (j > m ∨ A[i] ≤ B[j]):
        C[k] ← A[i], i ← i + 1
    else:
        C[k] ← B[j], j ← j + 1
return C[1...n + m]
```

3.3 归并排序

3.3 归并排序

分析下复杂度，发现是 $O(n+m)$ 的。那么怎么快速排好序呢？发现只需要对 $A[1...n], B[1...m]$ **分别调用一次排序**即可！

这就是**归并排序**（递归、合并）。

3.3 归并排序

分析下复杂度，发现是 $O(n+m)$ 的。那么怎么快速排好序呢？发现只需要对 $A[1...n], B[1...m]$ **分别调用一次排序**即可！

这就是**归并排序**（递归、合并）。

```
MergeSort(n, A[1...n]):
let C[1...n] ← {}
if n = 1:
    C[1...n] ← A[1...n]
else:
    MergeSort(⌊n/2⌋, A[1...⌊n/2⌋])
    MergeSort(⌈n/2⌉, A[⌊n/2⌋ + 1...n])
    C[1...n] ← Merge(⌊n/2⌋, ⌈n/2⌉, A[1...⌊n/2⌋], A[⌊n/2⌋ + 1...n])
return C[1...n]
```

3.3 归并排序

3.3 归并排序

老三样？这次我们可以继续**分析最坏复杂度**了。复杂度为 $T(n) = O(n) + 2T\left(\frac{n}{2}\right) = O(n \log n)$。

3.3 归并排序

老三样？这次我们可以继续**分析最坏复杂度**了。复杂度为 $T(n) = O(n) + 2T\left(\frac{n}{2}\right) = O(n \log n)$。

在此过程中，我们可以计算一些更有意思的东西，比如——**逆序对**。

P1908 逆序对

给定长为 n 的数组 $\{a_i\}$，求 $\#\{a_i > a_j \wedge i < j\}$。

$1 \le n \le 5 \times 10^5$。

3.3 归并排序

老三样？这次我们可以继续**分析最坏复杂度**了。复杂度为 $T(n) = O(n) + 2T\left(\frac{n}{2}\right) = O(n \log n)$。

在此过程中，我们可以计算一些更有意思的东西，比如——**逆序对**。

P1908 逆序对

给定长为 n 的数组 $\{a_i\}$，求 $\#\{a_i > a_j \wedge i < j\}$。

$1 \le n \le 5 \times 10^5$。

尝试在 i, j 合并时计算 (i, j) 是否是逆序对，发现每当**右侧序列首项比左侧序列首项小**时，出现左侧序列长度个逆序对。

这也证明了归并排序为什么可以低于 $O(n^2)$，因为它可以**在同一步内计算多个逆序对**。

3.4 快速排序

3.4 快速排序

还是考虑进行一些更远距离的交换操作，以便一次消除更多逆序对。

这里我们考虑每次**随机选取一个数**，讲比它小的数放在它的左侧，比它大的数放在它的右侧，随后进行分治解决规模更小的子问题。

3.4 快速排序

还是考虑进行一些更远距离的交换操作，以便一次消除更多逆序对。

这里我们考虑每次**随机选取一个数**，讲比它小的数放在它的左侧，比它大的数放在它的右侧，随后进行分治解决规模更小的子问题。

```
QuickSort(n, A[1...n]):
let C[1...n] ← {}
if n = 1:
    C[1...n] ← A[1...n]
else:
    let k ← Random(1, n)
    let p ← A[k]
    swap A[k] and A[n]
    let l ← 1, r ← n − 1
    while l < r:
```

```
        while A[l] ≤ p:
            l ← l + 1
        while A[r] > p:
            r ← r − 1
        if l < r:
            swap A[l] and A[r]
    swap A[l] and A[n]
    if l > 0:
        QuickSort(l − 1, A[1...l − 1])
        C[1...l − 1] ← A[1...l − 1]
    if l < n:
        QuickSort(n − l, A[l + 1...n])
        C[l + 1...n] ← A[l + 1...n]
return C[1...n]
```

老三样？

3.4 快速排序

3.4 快速排序

这里我们分析快速排序的复杂度，发现它的复杂度与**划分点的选取**有关。

如果每次划分点都选取**最小值**，那么复杂度为 $O(n^2)$。

一个**感性的想法**是，划分点的选取是均匀分布的，那么每次划分点都能将数组划分为大约 $\frac{n}{2}$ 的大小。

3.4 快速排序

这里我们分析快速排序的复杂度，发现它的复杂度与**划分点的选取**有关。

如果每次划分点都选取**最小值**，那么复杂度为 $O(n^2)$。

一个**感性的想法**是，划分点的选取是均匀分布的，那么每次划分点都能将数组划分为大约 $\frac{n}{2}$ 的大小。

我们仔细分析复杂度，有 $T(n) = O(n) + \frac{2}{n}\sum_{i=0}^{n-1} T(i)$。

断言 $T(n) \le cn\log n$，那么 $T(n) = O(n) + \frac{2}{n}\sum_{i=0}^{n-1} ci\log i$。

3.4 快速排序

3.4 快速排序

考虑边界情况 $T(0) = T(1) = c$，那么 $T(n) \le cn + \dfrac{2}{n}\sum_{i=0}^{n-1} ci\log i$。

对于 $n > 1$，我们有 $\sum_{i=0}^{n-1} i\log i \le \int_0^n x\log x\,\mathrm{d}x \le \frac{c}{2}n^2\log n - cn$。那么 $T(n) = O(n) + \dfrac{2}{n}\sum_{i=0}^{n-1} T(i) \le cn\log n$。

这样我们证明完成了 $T(n) = O(n\log n)$。

3.4 快速排序

考虑边界情况 $T(0) = T(1) = c$，那么 $T(n) \le cn + \frac{2}{n}\sum_{i=0}^{n-1} ci\log i$。

对于 $n > 1$，我们有 $\sum_{i=0}^{n-1} i\log i \le \int_0^n x\log x\,\mathrm{d}x \le \frac{c}{2}n^2\log n - cn$。那么 $T(n) = O(n) + \frac{2}{n}\sum_{i=0}^{n-1} T(i) \le cn\log n$。

这样我们证明完成了 $T(n) = O(n\log n)$。

考虑另一种方式分析复杂度，将复杂度贡献拆分到**每对交换**上。记排好序后的数组为 A'，交换 A'_i 和 A'_j 的概率为 $E_{i,j}$，可以进行如下分析：

i, j 会贡献当且仅当 i 或 j 是区间 $[i, j]$ 内最早出现的**划分点**，而 $[i, j]$ 中每个数是划分点的概率相同，故 $E_{i,j} = \frac{2}{j-i}$，那么复杂度为 $T(n) = \sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\frac{2}{j-i} = O(n\log n)$。

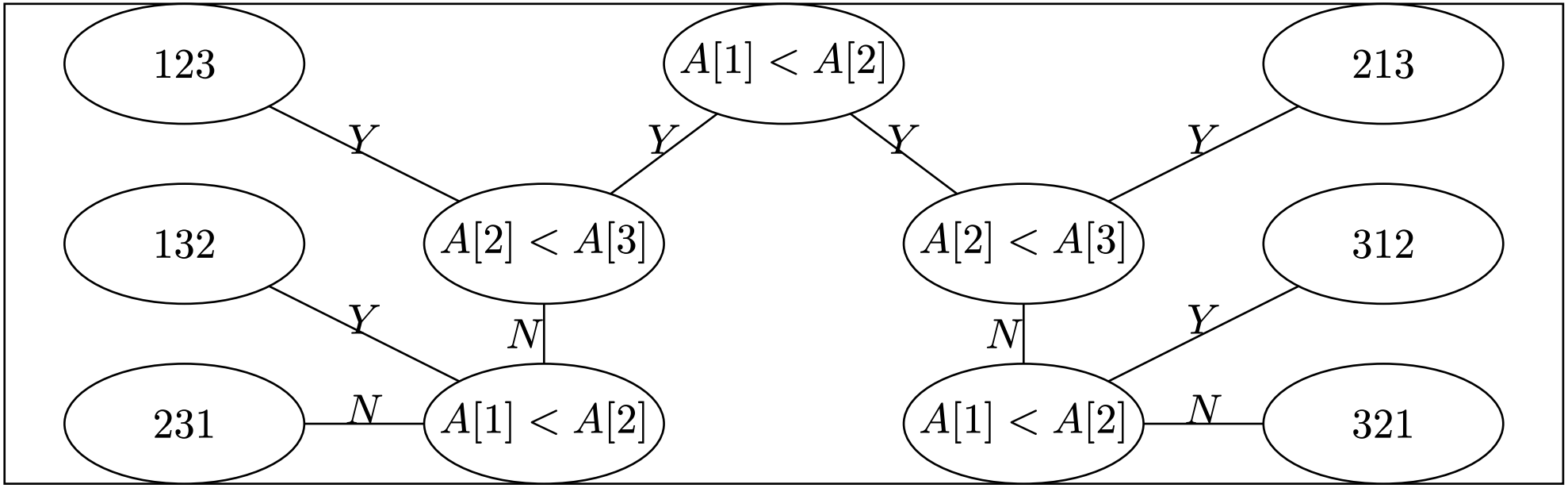
3.4 快速排序

3.4 快速排序

对于基于**比较**的排序算法，**平均复杂度**不会低于 $O(n \log n)$。

我们可以将整个算法在所有 $1 \sim n$ 的排列的比较次数作为**复杂度下界**，称为**决策树**。

下面是一份 $n = 3$ 的冒泡排序的决策树。



3.4 快速排序

3.4 快速排序

这里我们发现决策树是一棵**二叉树**，每个叶子节点代表一个**排列**，而每个非叶子节点代表一次**比较操作**。

考虑所以叶子的平均深度，**平均比较次数**为 $O(n \log n)$。

3.4 快速排序

这里我们发现决策树是一棵**二叉树**，每个叶子节点代表一个**排列**，而每个非叶子节点代表一次**比较操作**。

考虑所以叶子的平均深度，**平均比较次数**为 $O(n\log n)$。

证明比较容易。首先总节点数为 $2n!-1$，此时我们直接断言有 $\frac{n!}{2}$ 个叶子节点的深度**不小于** $\frac{1}{2}n\log n$。

考虑反证，假设有 $\frac{n!}{2}$ 个叶子节点的深度**不大于** $\frac{1}{2}n\log n$，那么总节点数的最大值为 $2^{\frac{1}{2}n\log n+1}=2\sqrt{n^n}$，最多只能有 $\sqrt{n^n}$ 个叶子，而 $\sqrt{n^n}=o\left(\frac{n!}{2}\right)$，矛盾！

那么**平均比较次数**不低于 $O(n\log n)$。

3.5 基数排序

3.5 基数排序

考虑当值域有点大时，**计数排序**的复杂度会很高。

那我们能不能通过一些方式减小值域呢？

3.5 基数排序

考虑当值域有点大时，**计数排序**的复杂度会很高。

那我们能不能通过一些方式减小值域呢？

3.5 基数排序

考虑当值域有点大时，**计数排序**的复杂度会很高。

那我们能不能通过一些方式减小值域呢？

```
RadixSort(n, A[1...n]):
let B[1...n] ← {}, V ← n
for i from 1 to d:
    for j from 1 to n:
        B[i] = ⌊A[i]/V^(i−1)⌋ mod d
    CountingSort(n, B[1...n])
return A[1...n]
```

老三样？

3.5 基数排序

考虑当值域有点大时，**计数排序**的复杂度会很高。

那我们能不能通过一些方式减小值域呢？

```
RadixSort(n, A[1...n]):
let B[1...n] ← {}, V ← n
for i from 1 to d:
    for j from 1 to n:
        B[i] = ⌊A[i]/V^(i−1)⌋ mod d
    CountingSort(n, B[1...n])
return A[1...n]
```

老三样？

这里我们发现值域的大小是 $\log_n V$，故复杂度为 $O(n \log_n V)$。

3.5 基数排序

3.5 基数排序

探讨一下基数排序的**适用范围**。

基数排序不能对**分数、有理数**进行排序。当我们发现**如果无法很有效的**划分关键字，那么基数排序无法使用。

3.5 基数排序

探讨一下基数排序的**适用范围**。

基数排序不能对**分数、有理数**进行排序。当我们发现**如果无法很有效的**划分关键字，那么基数排序无法使用。

真的吗？

分数排序

给定 n 个分数 $\left\{\dfrac{a_i}{b_i}\right\}$，将其在 $O(n)$ 时间内排序。

$1 \le n \le 5 \times 10^5, 1 \le a_i, b_i \le n^2$。

3.5 基数排序

探讨一下基数排序的**适用范围**。

基数排序不能对**分数、有理数**进行排序。当我们发现**如果无法很有效的**划分关键字，那么基数排序无法使用。

真的吗？

分数排序

给定 n 个分数 $\left\{\dfrac{a_i}{b_i}\right\}$，将其在 $O(n)$ 时间内排序。

$1 \le n \le 5 \times 10^5, 1 \le a_i, b_i \le n^2$。

将数值写为 $\left\lfloor \frac{a_i}{b_i} n^2 \right\rfloor$，我们可以发现之前**不等的数**现在**也不相同**，然后基数排序即可。

基数排序在**多关键字排序**中尤其常见，这点在后面接触**后缀数组**时还会再提及。

3.6 习题

3.6 习题

P1177 【模板】排序

P1966 [NOIP 2013 提高组] 火柴排队

P6186 [NOI Online 1 提高组] 冒泡排序

P9596 [JOI Open 2018] 冒泡排序 2 / Bubble Sort 2

P1138 第 k 小整数

P2127 序列排序

P2125 图书馆书架上的书

3.6 习题

3.6 习题

P4378 [USACO18OPEN] Out of Sorts S

P4375 [USACO18OPEN] Out of Sorts G

P4372 [USACO18OPEN] Out of Sorts P

P3819 松江 1843 路